Lab 3 Report: Symmetric Encryption and Hash Functions

Task 1: AES Encryption with Various Modes

Objective:

Encrypt a plaintext file using AES in three distinct modes (CBC, ECB, CFB) and confirm the encryption integrity by performing decryption on the resulting files.

Method:

Step 1: Created a text file named plain.txt with multiple lines of text content, then saved the file in the working directory

Step 2: Encryption using three different Modes

Commands Used:

CBC Encryption:

openssl enc -aes-128-cbc -e -in plain.txt -out cipher_cbc.bin -K 1a2b3c4d5e6f708192a3b4c5d6e7f80 -iv 1b2c3d4e5f607182

ECB Encryption:

openssl enc -aes-128-ecb -e -in input.txt -out encrypted-ecb.dat -K 1a2b3c4d5e6f708192a3b4c5d6e7f80

CFB Encryption:

openssl enc -aes-128-cfb -e -in input.txt -out encrypted-cfb.dat -K 1a2b3c4d5e6f708192a3b4c5d6e7f80 -iv 1b2c3d4e5f607182

Verification:

All output files were decrypted successfully with the matching decryption commands, validating the accuracy of the encryption procedure.

Decryption Commands:

CBC Decryption

openssl enc -aes-128-cbc -d -in encrypted-cbc.dat -out recovered-cbc.txt -K 1a2b3c4d5e6f708192a3b4c5d6e7f80 -iv 1b2c3d4e5f607182

ECB Decryption

openssl enc -aes-128-ecb -d -in encrypted-ecb.dat -out recovered-ecb.txt -K 1a2b3c4d5e6f708192a3b4c5d6e7f80

CFB Decryption

openssl enc -aes-128-cfb -d -in encrypted-cfb.dat -out recovered-cfb.txt -K 1a2b3c4d5e6f708192a3b4c5d6e7f80 -iv 1b2c3d4e5f607182

Output File

Output Files
- `encrypted-cbc.dat` - CBC encrypted output
- `encrypted-ecb.dat` - ECB encrypted output
- `encrypted-cfb.dat` - CFB encrypted output

Task 2: Encryption Mode — ECB vs CBC

Objective: To compare ECB and CBC encryption modes by encrypting a BMP image and observing the visual differences in the encrypted outputs

Method:

Step 1 — Setup

1. Converted the original image to BMP format using Paint
2. Created a working directory at: `E:\lab3\task3`
3. Changed PowerShell directory to the Task3 folder:

PS C:\Users\LENOVO> cd E:\lab3\task3

Step 2 — ECB Mode Encryption

Encryption command (run from PowerShell in `E:\lab3\task3`):

openssl enc -aes-128-ecb -e -in birdbmp.bmp -out bird_ecb.enc -K 00112233445566778899aabbccddeeff

Header Replacement Process :

1. Open the original `birdbmp.bmp` file in HxD hex editor.
2. Select **Edit** → **Select block**.
3. Copy the first 54 bytes.
4. Open the encrypted file `bird_ecb.enc` in HxD.
5. Select the first 54 bytes of the encrypted file.
6. Paste the copied header from the original `birdbmp.bmp`.
7. Save the modified file as `birdview_ecb.bmp`.

Step 3 — CBC Mode Encryption

Encryption command (run from PowerShell in `E:\lab3\task3`):

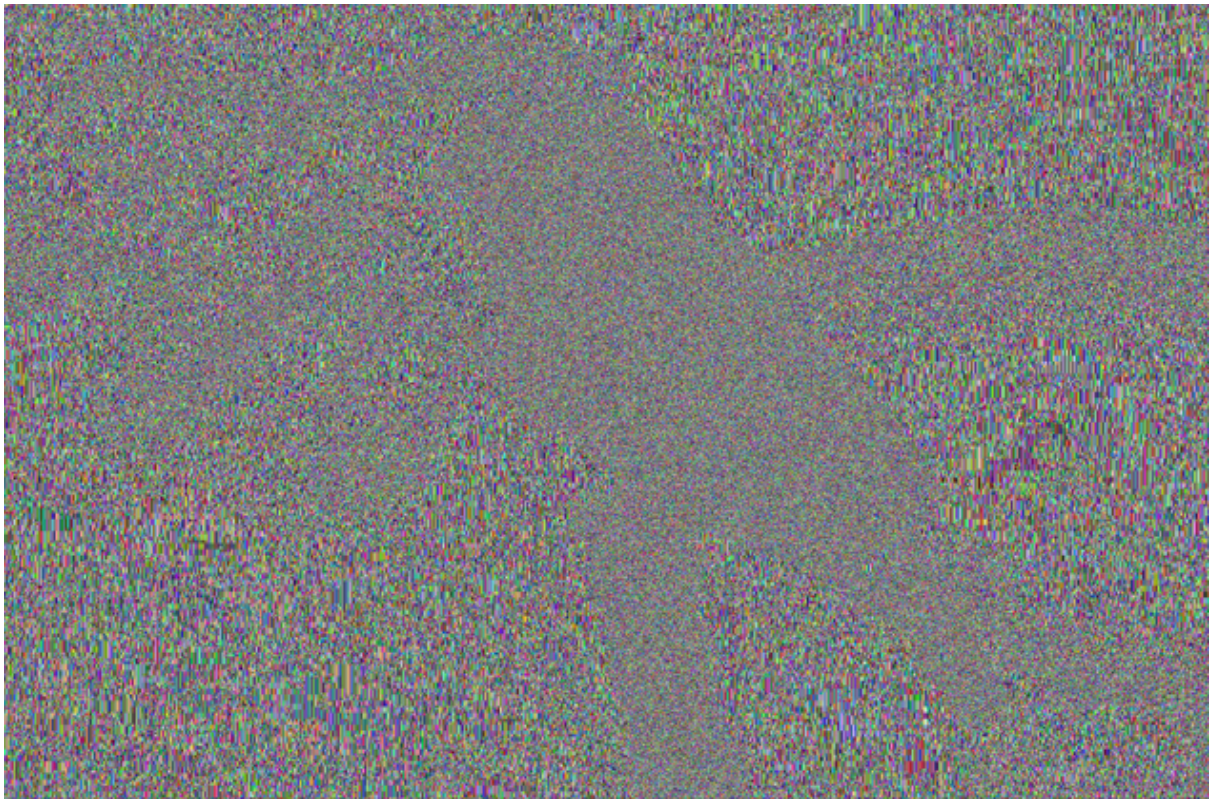
openssl enc -aes-128-cbc -e -in birdbmp.bmp -out bird_cbc.enc -K 00112233445566778899aabbccddeeff -iv 01020304050607080 90a0b0c0d0e0f10

Header Replacement Process :

1. Open the original `birdbmp.bmp` file in HxD hex editor.
2. Select and copy the first 54 bytes.
3. Open the encrypted file `bird_cbc.enc` in HxD.
4. Select the first 54 bytes of the encrypted file.
5. Paste the copied header from the original file.
6. Press **Ctrl+S** to save.
7. Save the file as `birdview_cbc.bmp`.

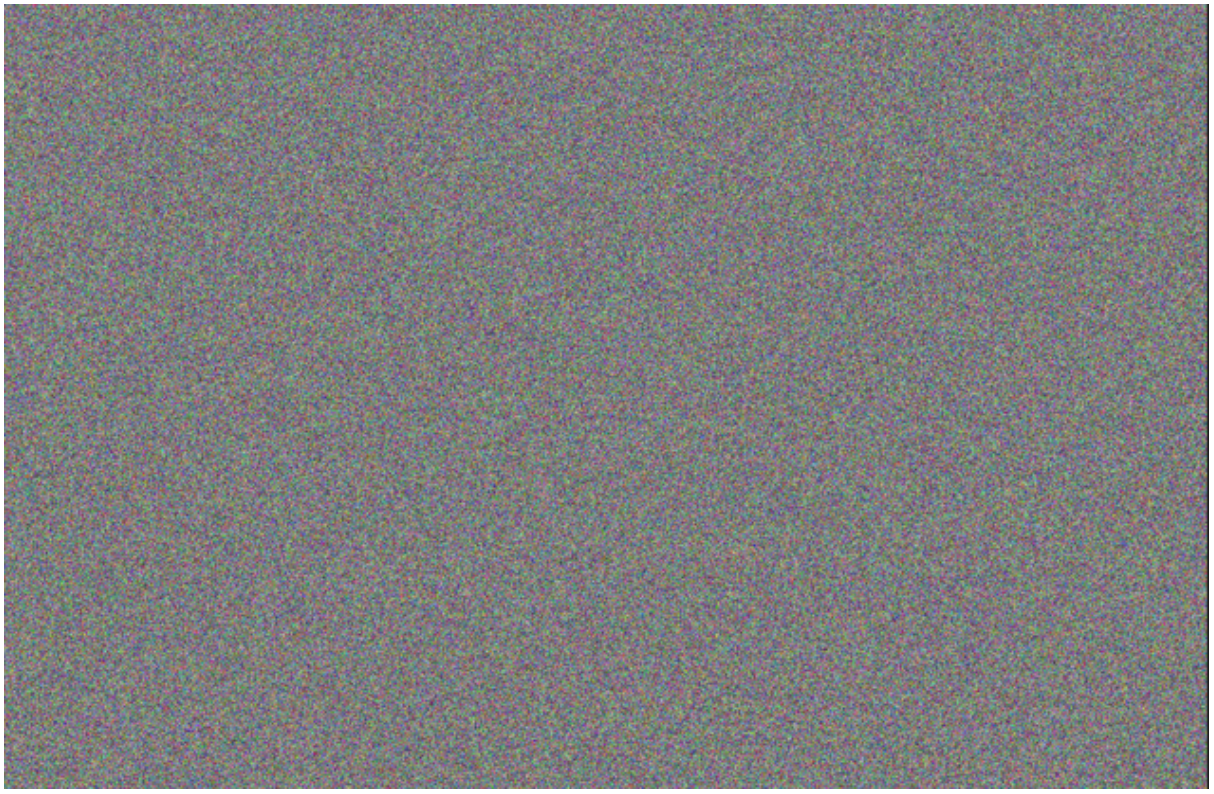
Results and Observations

ECB Mode Results (`birdview_ecb.bmp`)



1. The ECB-encrypted image reveals visible patterns from the original image.
2. Identical plaintext blocks produce identical ciphertext blocks.
3. The outline and structure of the original image remain visible (you can often see shapes, edges, or large color regions).

CBC Mode Results (`birdview_cbc.bmp`)



1. The CBC-encrypted image appears completely randomized.
2. No visible patterns or structure from the original image remain.
3. The image looks like random noise/static.
4. No useful visual information about the original image can be derived.

Analysis

Why ECB Mode is Insecure for Images

1. ECB encrypts each block independently without any chaining.
2. Identical plaintext blocks always produce identical ciphertext blocks.
3. Images often contain large areas of uniform or similar colors (repeating blocks).
4. These repetitive patterns are preserved in the encrypted output, revealing structural information.

5. An attacker can derive significant information about the image structure (outlines, shapes, repeated patterns).

Why CBC Mode is More Secure

1. CBC uses an Initialization Vector (IV) and chains blocks together.
2. Each plaintext block is XORed with the previous ciphertext block before encryption.
3. Identical plaintext blocks produce different ciphertext blocks because of chaining.
4. The chaining mechanism ensures that visual patterns are not preserved.
5. The encrypted output appears random, preventing leakage of image structure.

Security Implications

1. **ECB mode should never be used** for encrypting images or any data with visible or predictable patterns.
2. **CBC mode (with a random IV)** provides better confidentiality by hiding visual and structural patterns.
3. The choice of encryption mode significantly impacts security — chaining mechanisms (like CBC) are essential for semantic security.
4. Always use a secure IV (for CBC) and never reuse a key-IV pair across different messages.

Task 3: Encryption Modes – Altered Ciphertext

Objective

Examine error diffusion in encryption modes by altering one bit in the ciphertext and reviewing decryption outcomes.

Implementation

Encryption:

ECB:
openssl enc -aes-128-ecb -e -in plain.txt -out c_ecb.bin -K 00112233445566778899aabbccddeeff
CBC:

openssl enc -aes-128-cbc -e -in plain.txt -out c_cbc.bin -K 00112233445566778899aabbccddeeff -iv 01020304050607080 90a0b0c0d0e0f10

CFB:

openssl enc -aes-128-cfb -e -in plain.txt -out c_cfb.bin -K 00112233445566778899aabbccddeeff -iv 01020304050607080 90a0b0c0d0e0f10

OFB:

openssl enc -aes-128-ofb -e -in plain.txt -out c_ofb.bin -K 00112233445566778899aabbccddeeff -iv 01020304050607080 90a0b0c0d0e0f10

Decryption :

ECB:

openssl enc -aes-128-ecb -d -in c_ecb_corrupt.bin -out d_ecb_corrupt.txt -K 00112233445566778899aabbccddeeff

CBC:

openssl enc -aes-128-cbc -d -in c_cbc_corrupt.bin -out d_cbc_corrupt.txt -K 00112233445566778899aabbccddeeff -iv 01020304050607080 90a0b0c0d0e0f10

CFB:

openssl enc -aes-128-cfb -d -in c_cfb_corrupt.bin -out d_cfb_corrupt.txt -K 00112233445566778899aabbccddeeff -iv 01020304050607080 90a0b0c0d0e0f10

OFB:

openssl enc -aes-128-ofb -d -in c_ofb_corrupt.bin -out d_ofb_corrupt.txt -K 00112233445566778899aabbccddeeff -iv 01020304050607080 90a0b0c0d0e0f10

Analysis

- **For ECB: Only the corresponding block is corrupted** on decryption. Because each block is encrypted independently. Corruption doesn't propagate beyond that block.
- **For CBC:** The **current block and the next block become corrupted** on decryption. Each plaintext block is XOR-ed with the previous ciphertext block. So, one corrupted ciphertext block breaks its own decryption and corrupts the next block.
- **For CFB:** The corruption affects a **few bytes** but doesn't ruin the rest. CFB uses the previous ciphertext as input to the encryption step, so errors propagate for only a few bytes of plaintext, not indefinitely.
- **For OFB: Only one byte (or one block) is corrupted**. OFB generates a keystream independent of the plaintext; errors in ciphertext affect only matching bits on decryption, not future bytes. [Image comparing error diffusion in ECB, CBC, CFB, and OFB]

Reasons of Differences:

1. Error tolerance and data transmission

- Modes like **OFB** and **CFB** are better suited for streaming or communication channels where small bit errors might occur.
- **ECB** and **CBC** are not good for noisy channels since bit errors can ruin entire blocks.

2. Security implications

- **ECB leaks structure**. It's deterministic and should never be used for sensitive data like images.
- **CBC, CFB, and OFB hide patterns** much better.

3. Performance and parallelism

- **ECB** can be **parallelized easily**; others (especially CBC encryption) are sequential because of block dependencies.
- **OFB** and **CFB** behave like **stream ciphers** — useful for continuous data streams.

Task 4: Padding in AES Encryption Modes

Objective

The objective of this experiment is to observe how padding is applied in different AES encryption modes when the plaintext length is not a multiple of the AES block size (16 bytes). We also examine ciphertext sizes and verify the presence or absence of padding using a hex editor.

Tools Used

- **OpenSSL for Windows**
- **PowerShell**
- **HxD Hex Editor**

Plaintext File

A plaintext file named **`plain.txt`** was created.
The size of the plaintext was checked as:

Plaintext Size = 170 bytes

AES uses a fixed block size of 16 bytes.

To determine padding:

170 mod 16 = 10

Padding needed = 16 - 10 = 6 bytes
Padding byte value = 06 (hex)

So 6 padding bytes, each equal to `0x06`, are expected to appear in padded encryption modes.

Key and IV Used

Parameter	Value
AES Key (128-bit)	`00112233445566778899aabbccddeeff`
IV (for CBC/CFB/OFB)	`0102030405060708090a0b0c0d0e0f10`

Encryption Commands Executed

openssl enc -aes-128-ecb -in plain.txt -out cipher_ecb.bin -K 00112233445566778899aabbccddeeff
openssl enc -aes-128-cbc -in plain.txt -out cipher_cbc.bin -K 00112233445566778899aabbccddeeff -iv 0102030405060708090a0b0c0d0e0f10
openssl enc -aes-128-cfb -in plain.txt -out cipher_cfb.bin -K 00112233445566778899aabbccddeeff -iv 0102030405060708090a0b0c0d0e0f10
openssl enc -aes-128-ofb -in plain.txt -out cipher_ofb.bin -K 00112233445566778899aabbccddeeff -iv 0102030405060708090a0b0c0d0e0f10

Ciphertext File Sizes

File	Size (bytes)	Padding Present?	Explanation
`plain.txt`	170	—	Original plaintext
`cipher_ecb.bin`	**176**	**Yes**	ECB works on blocks → padding added
`cipher_cbc.bin`	**176**	**Yes**	CBC also works on blocks → padding added

`cipher_cfb.bin`	**170**	**No**	CFB operates like a stream → no padding
`cipher_ofb.bin`	**170**	**No**	OFB also operates like a stream → no padding

Conclusion so far:

ECB and CBC increase ciphertext size because padding is required.
CFB and OFB retain original size because no padding is required.

Padding Observation in HxD

The files **`cipher_ecb.bin`** and **`cipher_cbc.bin`** were opened in HxD.

At the end of both files, the following repeated byte pattern was observed:

06 06 06 06 06 06

This matches the expected PKCS#7 padding, since:

Padding length = 6 bytes → padding value = 0x06 repeated 6 times

The files `cipher_cfb.bin` and `cipher_ofb.bin` showed no repeating padding values. Their final bytes appeared random, confirming no padding was applied.

Decryption Verification

All ciphertexts were decrypted using OpenSSL.

Example decryption command:

openssl enc -aes-128-cbc -d -in cipher_cbc.bin -out decrypted_cbc.txt -K 00112233445566778899aabbccddeeff -iv 01020304050607080 90a0b0c0d0e0f10

All decrypted files were compared with the original plaintext and matched exactly, confirming correct padding and unpadding operation.

Final Conclusion

- AES-128-ECB and AES-128-CBC use PKCS#7 padding because they operate on fixed 16-byte blocks.
  When the plaintext size is not a multiple of 16, padding bytes are added.
  In our case, 170 → 176 bytes, with padding byte = `0x06`.

- AES-128-CFB and AES-128-OFB operate in stream mode, so no padding is required.
  Ciphertext size remains equal to plaintext size .

Therefore, padding is required only in ECB and CBC, but not in CFB and OFB.

Task 5: Message Digest Generation

Objective:

Produce digests via multiple hash methods and assess their results.

Implementation:

Plaintext file content:
Message for digest purpose

Commands Used:

MD5 → openssl dgst -md5 plain.txt > md5.txt
SHA1 → openssl dgst -sha1 plain.txt > sha1.txt
SHA256 → openssl dgst -sha256 plain.txt > sha256.txt
SHA512 → openssl dgst -sha512 plain.txt > sha512.txt

Generated Digests:

MD5:
MD5(input.txt)= e777ebb98c59af9fd4142685c597d154

- Length: 128 bits (32 hex)

- Quick but insecure (collision risks)

SHA1:
SHA1(input.txt)= 87ec3acf9ba6542b6447fd8085ac10119491816b

- Length: 160 bits (40 hex)

- Phased out for vulnerabilities

SHA256:
SHA2-256(input.txt)=
939106512d73323c0164c0b3761441c10fcaa3020ffc3692b6fe68dcf67b1221

- Length: 256 bits (64 hex)
- Secure and standard

SHA512:
SHA2-512(input.txt)=
76450fe383aa94624ea48dee3a2f355b247002075f49fd6c2178bb5580dbe49bdf97764099ef
b8ae6ee1bbc8e8d20348894b4b5b7d504555c67e72a79a8d952

- Length: 512 bits (128 hex)
- Maximal strength for critical use

Observations:

1. Fixed Output: Uniform size irrespective of input
2. Cascade Impact: Minor input tweaks cause major hash changes
3. Consistency: Same input always gives same hash
4. Irreversibility: Hash cannot be reversed
5. Viability: SHA256/SHA512 strong; MD5/SHA1 outdated

Output Files:

md5.txt – MD5 result
sha1.txt – SHA1 result
sha256.txt – SHA256 result
sha512.txt – SHA512 result

Task 6: Keyed Hashing and HMAC

Objective:

Compute HMAC with different hash types and key sizes.

Implementation:

Plaintext:
Message for digest purpose

A bash script tested HMAC for multiple key lengths.

HMAC Outputs

HMAC-MD5:
Key “a” (1 byte) → fdef49ee857fd2a6408b394faaf9d761
Key “abcdefg” (7 bytes) → 16f0cfb70d107a98165878d892f30580
Key “0123456789abcdef” (16 bytes) → a71d32856507edffe52c09b842ee912d
Key “This is a longer key…” (89 bytes) → b9f2eed319b6a10701fd9db515d14a12

HMAC-SHA1:
Key “a” → fba55873d89bdaedfef135f0db2ba6192229fbe3
Key “abcdefg” → 398e52807937514239874edce6b1315df732a7fe
Key “0123456789abcdef” → 1af5cf1babe3a3f0ee150db539f56622154fddca
Key “This is a longer key…” → dadd43f4492b8bfb81c2dcdb95868a9f9e4d50ee

HMAC-SHA256:
Key “a” → 9450cfe629e371c30f171d1b08e1df4176ce770da8eccf9eef23225f51edd73a
Key “abcdefg” →
98eb0d1d223e721458eb908216e75e3b07ae721ab34c426e4bda322f628d3211
Key “0123456789abcdef” →
c0f75f0d9e7b512abfd0c4119026bd99f0ed2e11c98d63176bdacc25ff28c070
Key “This is a longer key…” →
08f1cfebb9b987f2c35f68f5ab6db481f31c9c40176fcb25ce1b765606f8b4e4

Key Sizing Evaluation:

- No fixed key size needed
- Short keys are zero-padded
- Long keys are hashed down
- Recommended: longer keys, ideally ≥ output size (e.g., 32 bytes for SHA256)

Task 7: Hash Function Characteristics

Objective:

Show diffusion by changing one bit in input and comparing hashes.

Implementation:

Original (input.txt):
Message for hash testing
Author: Zahid Hasan

Commands:
Generate hash for original file:
openssl dgst -md5 plain.txt > plain_md5.txt
openssl dgst -sha256 plain.txt > plain_sha256.txt

Generate hash for modified file (one bit flipped):
openssl dgst -md5 modified.txt > modified_md5.txt
openssl dgst -sha256 modified.txt > modified_sha256.txt

Hash Outputs

MD5:
Original: e635e11d8d684e7f1a3db2e349a873a4
Altered: 54ed6cd13b13b327784b183bc42440a0

SHA256:
Original: 47c7c246016a6a12c4218ac150bb71c4eb177bf303e6c30563b61a54bb56dac2
Altered: ec0fc6cf8ead84f0e38df808ac17dd47638e1e1826ddf52d8ecc6aad3ee8478c

Observations:

1. Huge change from a 1-bit modification
2. Around 50% bit difference expected
3. Outputs show no similarity
4. Same input = same hash

Bonus: Bit Difference Script

Python program comparing matching bits between two hashes.

Results:

MD5: 62 / 128 bits match (~48.44%)
SHA256: 123 / 256 bits match (~48.05%)

Evaluation:

~50% difference confirms proper diffusion. Good for tamper detection.

Output Files:

plain_md5.txt – Hash of original file
plain_sha256.txt – SHA256 hash of original file
modified_md5.txt – Hash of modified file

modified_sha256.txt – SHA256 hash of modified file
compare.py – Python bit-comparison script